Manual Técnico – TechCare Solutions

Este manual describe cómo está construido el sistema de servicio técnico, inventario y facturación de TechCare Solutions: su arquitectura, las tecnologías usadas, la estructura del proyecto, cómo configurarlo, instalarlo y ejecutarlo, y los componentes de la base de datos, que es el núcleo de la solución.

1. Arquitectura

El sistema usa una arquitectura de tres capas. La interfaz web solo se comunica con la API por HTTP; la API es la única que se conecta a SQL Server, y lo hace llamando a procedimientos almacenados, vistas y funciones. Las reglas del negocio importantes (stock, saldos, estados, precios históricos) están en la base de datos, de modo que se cumplen aunque alguien intente insertar datos directamente en SQL Server (R16).

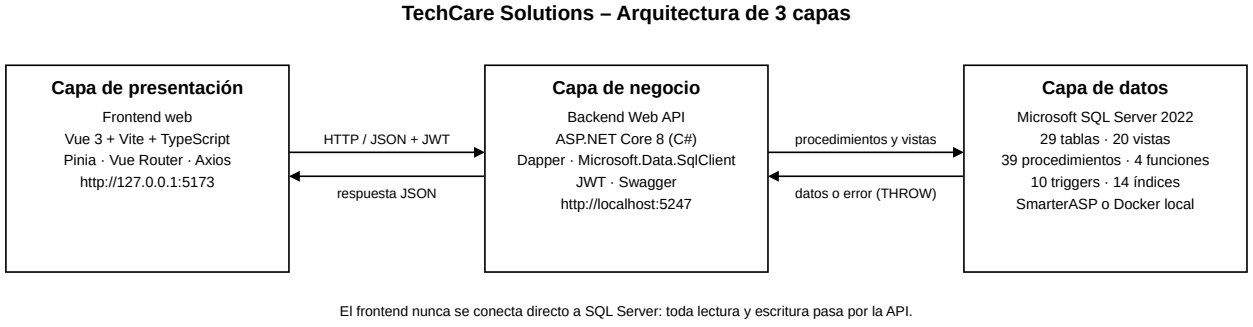


Figura 1. Arquitectura de 3 capas

Flujo de una petición

1. El usuario inicia sesión con su correo; la API ejecuta `sp_LoginUsuario`, que compara el hash SHA-256 y devuelve roles y permisos.
2. La API firma un token JWT con el rol y el frontend lo envía en cada petición (`Authorization: Bearer`).
3. La API valida la política del rol y ejecuta el procedimiento con parámetros (Dapper), nunca concatenando texto del usuario.
4. Si el procedimiento detecta un error de negocio, hace ROLLBACK y lanza `THROW`; la API lo devuelve como respuesta 400 con un mensaje claro.

2. Tecnologías

Componente	Tecnología y versión	Uso
Base de datos	Microsoft SQL Server 2022 (16.x)	Motor obligatorio; en la nube en SmarterASP (sql5113.site4now.net) o local en Docker.
Backend	.NET 8 · ASP.NET Core Web API (C#)	Capa de servicios con las reglas del negocio.
Acceso a datos	Dapper 2.1 · Microsoft.Data.SqlClient 7.1	Consultas parametrizadas y ejecución de procedimientos almacenados.
Seguridad	JWT Bearer (Microsoft.AspNetCore.Authentication.JwtBearer 8.0)	Token con el rol del usuario; políticas por rol.
Documentación API	Swagger (Swashbuckle 10.3)	Prueba de los endpoints en /swagger.

Frontend	Vue 3.5 · Vite · TypeScript 5.9	Interfaz web.
Librerías frontend	Pinia, Vue Router, Axios, VeeValidate + Zod, Tailwind CSS 4, Lucide	Estado, rutas, llamadas HTTP, validación de formularios y estilos.
Herramientas	SSMS 20+, Visual Studio 2022, VS Code, Node.js 22.12+, Docker Desktop, GitHub Desktop	Desarrollo, ejecución de scripts y control de versiones.

Lenguajes utilizados

Lenguaje	Capa	Dónde se usa
T-SQL (Transact-SQL)	Datos	Scripts de la carpeta database: tablas y restricciones, procedimientos almacenados, funciones, vistas, triggers, índices y datos de prueba.
C#	Negocio	API en ASP.NET Core 8: controladores, DTO, servicio JWT y conexión con Dapper.
TypeScript	Presentación	Lógica del frontend: rutas, stores de Pinia, llamadas a la API con Axios y validaciones con Zod.
HTML y CSS (componentes Vue)	Presentación	Plantillas y estilos de cada pantalla en archivos .vue, con Tailwind CSS.

Formatos de apoyo: JSON (configuración, secretos y el detalle de compras que `sp_RegistrarCompra` lee con `OPENJSON`), YAML (`docker-compose.yml`) y Markdown (README).

3. Estructura del proyecto

```
ProyectoBaseDeDatosI/
├── BD/BD_Conetner_BD1/        docker-compose.yml de SQL Server 2022 (puerto 1441)
├── database/                  scripts SQL
│   ├── 00_script_maestro.sql  crea TODA la base (scripts 01 al 12 en orden)
│   ├── 01_tablas.sql          27 tablas con PK, FK, UNIQUE, DEFAULT y CHECK
│   ├── 02_datos_iniciales.sql catálogos y usuarios base
│   ├── 03_procedimientos.sql  registro de orden, cambio de estado y login
│   ├── 04_procesos.sql        diagnóstico, reparación, repuestos, compras y vistas
│   ├── 05_datos_prueba.sql    datos de prueba (mínimos de la sección 7 del enunciado)
│   ├── 06_facturacion.sql     facturación, pagos y entrega
│   ├── 07_triggers_indices_reportes.sql
│   │                          triggers, índices y reportes
│   ├── 08_permisos.sql        Permiso y RolPermiso (permisos por rol)
│   ├── 09 al 12               vistas y SP de listados y mantenimientos
│   └── 0X_pruebas_*.sql       pruebas de cada script
├── Backend_PFDB1/Backend_API_Proyevto_Base_de_Datos_I/
│   ├── Program.cs             configuración: JWT, políticas por rol, CORS, Swagger
│   ├── Controllers/           12 controladores (Auth, Cliente, Equipo, Orden, Taller,
│   │                          Inventario, Caja, Reportes, Mantenimiento, Usuario,
│   │                          Permiso, Test)
│   ├── DTO/                   llaman a procedimientos y vistas con Dapper
│   ├── Services/JwtService.cs genera el token
│   └── BD/DBContext.cs        abre la conexión (cadena en secretos de usuario)
├── Frontend_PFDB1/
│   ├── src/app/               App.vue, router y vistas generales (inicio/dashboard)
│   ├── src/core/              cliente HTTP (Axios), configuración (.env) y utilidades
│   ├── src/shared/            componentes, layouts y estilos compartidos
│   ├── src/features/          un módulo por área: auth, clientes, equipos, ordenes,
│   │                          taller, inventario, caja, reportes, mantenimientos,
│   │                          usuarios
│   └── .env.example           VITE_API_BASE_URL=http://localhost:5247
├── docs/                      ER, relacional, normalización, pruebas y manuales
└── README.md                  pasos exactos de instalación
```

4. Configuración e instalación

4.1 Requisitos

Git o GitHub Desktop, .NET SDK 8.0, Node.js 22.12 o superior, SQL Server Management Studio 20 (o VS Code con la extensión mssql) y, solo si la base va en local, Docker Desktop.

4.2 Base de datos

La base se crea completa con un solo archivo, `database/00_script_maestro.sql`, que ejecuta los scripts 01 al 12 en el orden de sus dependencias: tablas y restricciones, catálogos, permisos, procedimientos, procesos, datos de prueba, facturación, triggers e índices, y mantenimientos. El script borra las tablas si ya existen.

- **Local (Docker):** `cd BD/BD_Conetner_BD1`, copiar `.env.example` a `.env` con una contraseña para `sa` y ejecutar `docker compose up -d`. En SSMS conectarse a `localhost,1441`, quitar el comentario a las líneas `CREATE DATABASE TechCare` / `USE TechCare` y ejecutar todo.
- **Nube (SmarterASP):** conectarse al servidor con el usuario de la base, seleccionar la base y ejecutar el script con esas líneas comentadas.

Verificación:

```
SELECT type_desc, COUNT(*) AS Cantidad
FROM sys.objects
WHERE type IN ('U','V','P','FN','IF','TF','TR')
GROUP BY type_desc;
-- 29 tablas, 20 vistas, 39 procedimientos, 10 triggers y 4 funciones
```

4.3 Backend

Las credenciales no están en `appsettings.json` (sus valores quedan vacíos). Se guardan con los **secretos de usuario** de .NET, fuera del repositorio, en `%APPDATA%\Microsoft\UserSecrets\techcare-backend-pfdb1\secrets.json`:

```
cd Backend_PFDB1/Backend_API_Proyevto_Base_de_Datos_I
dotnet user-secrets set 'ConnectionStrings:DefaultConnection' 'Server=...;Database=...;User
Id=...;Password=...;TrustServerCertificate=True;'
dotnet user-secrets set 'Jwt:Key' '<clave aleatoria de 32 caracteres o más>'
dotnet run
```

ASP.NET Core lee la configuración en este orden: `appsettings.json` → `appsettings.Development.json` → secretos de usuario → variables de entorno. Si falta un valor, el backend se detiene con un mensaje que indica cuál falta. Swagger queda en `http://localhost:5247/swagger` y `GET /api/test/conexion` debe responder `"conectado": true`.

4.4 Frontend

```
cd Frontend_PFDB1
copy .env.example .env      # VITE_API_BASE_URL=http://localhost:5247
npm install
npm run dev                 # http://127.0.0.1:5173
```

El backend debe estar corriendo antes de iniciar sesión. La API solo acepta llamadas del frontend desde `localhost:5173` y `127.0.0.1:5173` (CORS).

4.5 Usuarios de prueba

Todos usan la contraseña `TechCare2026!` (en la base solo se guarda su hash). Se inicia sesión con el correo: `admin`, `recepcion1`, `tecnico1`, `bodega1`, `caja1`, etc., con el dominio `@techcare.com.gt`. La lista completa está en el README.

5. Seguridad

- **Contraseñas:** se guardan como hash SHA-256 (`HASHBYTES('SHA2_256', …)`); nunca en texto plano.
- **Roles y permisos:** relación formal Usuario–UsuarioRol–Rol y permisos modelados en Permiso y RolPermiso. Los cinco roles son ADMINISTRADOR, RECEPCION, TECNICO, BODEGA y CAJA.
- **Autorización:** la API usa políticas por rol sobre el token JWT, y los procedimientos vuelven a validar el rol con `fn_UsuarioTieneRol` (R03).
- **Consultas parametrizadas:** todo pasa por Dapper con parámetros, lo que evita la inyección SQL.
- **Variables sensibles:** cadena de conexión y clave JWT en secretos de usuario; `.env`, `secrets.json` y respaldos `.bak` no se suben al repositorio.
- **Validación en dos lados:** el frontend valida formularios con VeeValidate + Zod y la API valida de nuevo con anotaciones y en los procedimientos.

Endpoints principales y roles

Ruta	Operaciones	Roles autorizados
`/api/Auth`	login, perfil	Público (login)
`/api/Cliente`, `/api/Equipo`	listar, crear, actualizar, desactivar, activar	ADMINISTRADOR, RECEPCION
`/api/Orden`	listar, detalle, historial, registrar, cambiar estado	Ver: todos · Crear: ADMIN, RECEPCION · Estado: ADMIN, RECEPCION, TECNICO
`/api/Taller`	órdenes del taller, diagnóstico, iniciar/finalizar reparación, actividades, repuestos	ADMINISTRADOR, TECNICO
`/api/Inventario`	stock, movimientos, compras (registrar, recibir, anular)	ADMINISTRADOR, BODEGA
`/api/Caja`	órdenes por cobrar, facturar, pagos, entregar	Facturar y cobrar: ADMIN, CAJA · Entregar: ADMIN, RECEPCION
`/api/Reportes`	reportes 1 al 12 y dashboard	ADMINISTRADOR
`/api/Mantenimiento`	repuestos, proveedores, bodegas, categorías, tipos de equipo, mínimos	ADMINISTRADOR
`/api/Usuario`, `/api/Permiso`	usuarios, roles y permisos	ADMINISTRADOR (Permiso/mios: usuario autenticado)
`/api/Test`	prueba de conexión con SQL Server	Público

6. Componentes de la base de datos

Objeto	Cantidad	Dónde se crea
Tablas	29	01 (27) y 08 (2)
Restricciones (PK, FK, UNIQUE, CHECK, DEFAULT)	160	01 (153) y 08 (7)
Índices no agrupados	14	01 (1) y 07 (13)
Vistas	20	04, 07, 08, 09 y 10
Funciones	4	04 (3) y 07 (1)
Procedimientos almacenados	39	03, 04, 06, 07, 09, 10, 11 y 12
Triggers	10	07

6.1 Tablas por módulo

Módulo	Tablas
Seguridad	Usuario, Rol, UsuarioRol, Permiso, RolPermiso
Clientes y equipos	Cliente, TipoEquipo, Equipo

Órdenes de servicio	OrdenServicio, EstadoOrden, HistorialEstadoOrden, Entrega
Diagnóstico y reparación	Diagnostico, Reparacion, ActividadReparacion, ReparacionRepuesto
Repuestos e inventario	Categoria, Repuesto, AuditoriaPrecio, Bodega, Inventario, MovimientoInventario
Proveedores y compras	Proveedor, ProveedorRepuesto, Compra, DetalleCompra
Facturación y pagos	Factura, Pago, MetodoPago

Relaciones N:M resueltas con tablas asociativas: UsuarioRol, RolPermiso, Inventario, ProveedorRepuesto, DetalleCompra y ReparacionRepuesto. Llaves primarias compuestas: esas seis más ActividadReparacion. Todas las llaves foráneas usan ON DELETE NO ACTION y ON UPDATE NO ACTION, porque la información transaccional no se borra (R14). El detalle de columnas está en el modelo relacional (`docs/Modelo_Relacional_TechCare.pdf`) y la justificación del diseño en el documento de normalización.

6.2 Procedimientos almacenados

Los procesos del negocio usan `BEGIN TRANSACTION` con `TRY…CATCH`: si cualquier paso falla se hace `ROLLBACK` y no queda ningún cambio parcial (R15).

Procedimiento	Qué hace	Transacción
`sp_LoginUsuario`	Valida correo y hash SHA-256; devuelve el usuario, sus roles y sus permisos.	No
`sp_RegistrarOrden`	Crea la orden con número único, estado RECIBIDO y su primer historial.	Sí
`sp_CambiarEstadoOrden`	Cambia el estado validando transiciones y guarda el historial (R05).	Sí
`sp_RegistrarDiagnostico`	Registra el diagnóstico (técnico autorizado, R03) y pasa la orden a ESPERANDO_AUTORIZACION.	Sí
`sp_IniciarReparacion`	Exige diagnóstico previo (R04), crea la reparación y pasa la orden a REPARACION.	Sí
`sp_RegistrarActividadReparacion`	Agrega una actividad a la reparación.	Sí
`sp_AsignarRepuesto`	Valida stock (R06), guarda el precio histórico, descuenta inventario y crea la SALIDA (R07).	Sí
`sp_FinalizarReparacion`	Cierra la reparación con su mano de obra y deja la orden LISTO.	Sí
`sp_RegistrarCompra`	Crea la compra (encabezado y detalle desde JSON) en estado PENDIENTE.	Sí
`sp_RecibirCompra`	Suma inventario y crea una ENTRADA por línea (R08); @SimularError demuestra el ROLLBACK (T07).	Sí
`sp_AnularCompra`	Anula una compra pendiente sin borrarla (R14).	Sí
`sp_GenerarFactura`	Factura una orden LISTO: mano de obra + repuestos, IVA 12 %; máximo una por orden (R11).	Sí
`sp_RegistrarPago`	Registra un pago sin exceder el saldo (R12); marca PAGADA cuando el saldo llega a 0.	Sí
`sp_EntregarOrden`	Registra la entrega solo con la factura pagada y pasa la orden a ENTREGADO.	Sí
`sp_TrazabilidadOrden`	Reporte 12: resumen y línea de tiempo de una orden.	No
`sp_ListarOrdenes`	Listado de órdenes con filtros por texto y estado.	No
`24 SP de mantenimiento`	Listar, crear, actualizar, activar y desactivar clientes, equipos, repuestos, proveedores, bodegas, categorías, tipos de equipo y usuarios.	Los que escriben

6.3 Funciones

Función	Tipo	Qué calcula
`fn_SaldoFactura`	Escalar	Saldo = total de la factura − pagos APLICADOS (R13).

fn_TotalOrden	Escalar	Total de la orden sin IVA = mano de obra + repuestos a precio histórico.
fn_UsuarioTieneRol	Escalar	Indica si el usuario activo tiene el rol pedido; valida R03 dentro de los procesos.
fn_ReporteIngresos	De tabla	Reporte 11 para cualquier rango de fechas.

6.4 Vistas

Vista	Uso	Contenido
vw_OrdenesActivas	Reporte 1	Órdenes activas por estado, prioridad y técnico (LEFT JOIN: una orden sin diagnóstico no desaparece).
vw_HistorialCliente	Reporte 2	Historial de órdenes de un cliente (LEFT JOIN a factura y entrega).
vw_HistorialEquipo	Reporte 3	Historial técnico de un equipo por número de serie.
vw_TrabajoTecnicos	Reporte 4	Diagnósticos y reparaciones por técnico y fecha.
vw_RepuestosUtilizados	Reporte 5	Repuestos usados con costo y venta históricos y el precio actual para comparar.
vw_InventarioActual	Reporte 6	Existencia por bodega, categoría y repuesto, con su valor.
vw_RepuestosBajoMinimo	Reporte 7	Repuestos con existencia igual o menor al mínimo.
vw_ComprasProveedor	Reporte 8	Compras por proveedor con detalle y costo histórico.
vw_MovimientosInventario	Reporte 9	Movimientos con documento origen (compra u orden) y usuario.
vw_FacturasSaldoPendiente	Reporte 10	Facturas con saldo y el detalle de sus pagos.
vw_IngresosPeriodo	Reporte 11	Ingresos por mes separando mano de obra y repuestos.
vw_TrazabilidadOrden	Reporte 12	Resumen completo de una orden de cliente a pagos.
vw_Dashboard	Inicio	Números de la pantalla principal (órdenes por estado, saldos, bajo mínimo).
vw_ActividadReciente	Inicio	Cambios de estado, movimientos y pagos más recientes.
vw_Cliente, vw_Equipo, vw_Orden	Listados	Listados de clientes, equipos y órdenes con sus nombres.
vw_UsuarioLista	Usuarios	Usuarios con sus roles.
vw_Permiso, vw_UsuarioPermiso	Seguridad	Permisos por rol y por usuario (Usuario → UsuarioRol → RolPermiso → Permiso).

6.5 Triggers

Trigger	Evento	Protección
trg_Repuesto_AuditoriaPrecio	AFTER UPDATE	Guarda en AuditoriaPrecio cada cambio de precio de compra o venta.
trg_Pago_NoExcederTotal	AFTER INSERT, UPDATE	La suma de pagos nunca pasa del total de la factura (R12), aunque se inserte directo en SQL.
trg_Entrega_SinSaldo	AFTER INSERT	No se entrega un equipo sin factura o con saldo pendiente.
trg_OrdenServicio_EntregadoConEntrega	AFTER UPDATE	Una orden solo pasa a ENTREGADO si existe su entrega.
trg_OrdenServicio_NoEliminar, trg_HistorialEstadoOrden_NoEliminar, trg_MovimientoInventario_NoEliminar, trg_Compra_NoEliminar, trg_Factura_NoEliminar, trg_Pago_NoEliminar	AFTER DELETE	Impiden borrar información transaccional (R14): lanzan THROW y el DELETE se revierte; se usa estado, cancelación o anulación.

6.6 Índices no agrupados

Índice	Tabla (columnas)	Justificación

UX_Cliente_NIT	Cliente (NIT)	Único filtrado: el NIT no se repite, pero "CF" sí puede repetirse.
IX_OrdenServicio_Estado_Prioridad	OrdenServicio (EstadoOrdenId, Prioridad)	Reporte 1 y dashboard: órdenes activas por estado y prioridad.
IX_OrdenServicio_Equipo_Fecha	OrdenServicio (EquipoId, FechaIngreso)	Reporte 3: historial de un equipo.
IX_OrdenServicio_Cliente_Fecha	OrdenServicio (ClienteId, FechaIngreso)	Reporte 2: historial de un cliente.
IX_HistorialEstadoOrden_Orden_Fecha	HistorialEstadoOrden (OrdenId, FechaCambio)	Historial y trazabilidad de una orden.
IX_Diagnostico_Tecnico_Fecha	Diagnostico (TecnicoId, FechaDiagnostico)	Reporte 4: trabajo por técnico en un rango de fechas.
IX_Reparacion_Tecnico_Fecha	Reparacion (TecnicoId, FechaInicio)	Reporte 4: trabajo por técnico en un rango de fechas.
IX_MovimientoInventario_Fecha	MovimientoInventario (FechaMovimiento)	Reporte 9 y actividad reciente.
IX_MovimientoInventario_Repuesto_Bodega	MovimientoInventario (RepuestoId, BodegaId, FechaMovimiento)	Kárdex de un repuesto en una bodega.
IX_Compra_Proveedor_Fecha	Compra (ProveedorId, FechaCompra)	Reporte 8: compras por proveedor.
IX_Factura_FechaEmision	Factura (FechaEmision)	Reporte 11: ingresos por período.
IX_Pago_Factura	Pago (FacturaId, Estado)	Cálculo del saldo (fn_SaldoFactura) y reporte 10.
IX_Equipo_Cliente	Equipo (ClienteId)	Equipos de un cliente al registrar una orden.
IX_Cliente_Nombre	Cliente (NombreRazonSocial)	Búsqueda de clientes por nombre.

6.7 Respaldo

En SmarterASP la copia se genera desde el panel (Bases de datos → MSSQL → Copia de seguridad) y se descarga desde Archivos → `db`. En local se usa `BACKUP DATABASE TechCare TO DISK = ….` Como el script maestro recrea toda la base, también sirve para reconstruirla desde cero en cualquier SQL Server.

7. Problemas comunes

Mensaje	Causa	Solución
Jwt:Key no está configurada / No se encontró la cadena de conexión	Faltan los secretos de usuario	Configurarlos (4.3)
Network Error al iniciar sesión	El backend no está corriendo	Ejecutar dotnet run o ▶ en Visual Studio
Login failed for user … (error 18456)	Contraseña de la base incorrecta o cuenta del hosting suspendida	Corregir secrets.json o revisar el estado de la base en el panel
Correo o contraseña incorrectos	Se escribió el usuario en vez del correo, o el usuario está inactivo	Usar el correo completo
La base ya tiene órdenes o datos de prueba	Se ejecutó el 05 solo sobre una base con datos	Ejecutar el 00_script_maestro.sql completo